

Design Document

Group Monitoring System for Mobile IoT Devices

Simone Mannarino, Alessandro Martinolli, Andrea Paparella

September 6, 2024

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Project Overview	3
1.3	Assumptions and Guidelines	3
2	System Architecture	4
2.1	High-Level Architecture Diagram	4
2.2	Communication Protocols	4
3	IoT Simulation Environment (Contiki, Cooja)	4
3.1	System Constraints	4
3.2	System Implementation	4
3.3	Limitations	5
3.4	System Scenario Overview	5
4	Integration & Processing Backend (Node-RED)	6
4.1	High Level Architecture	6
4.2	Design Choices	6
4.3	Implementation of Group Monitoring	6
4.3.1	MQTT Communication Nodes	6
4.3.2	Function Nodes for Data Processing	7
4.3.3	Periodic Computation	8
4.4	Integration with Apache Spark Streaming	8
5	Analytics Backend (Spark)	9
5.1	High-Level Architecture Diagram	9
5.2	Design Choices	9
5.3	Component Description	9
5.3.1	Entry Point	9
5.4	Flask Server	9
5.5	Tracker App	11
5.6	Top-N App	12
5.6.1	Functionality Overview	12
6	Conclusion	12
6.1	Summary	12
6.2	Future Work	12

1 Introduction

1.1 Purpose

This document describes the design and implementation of a group monitoring system for Mobile IoT devices, focusing on group formation and dynamic membership reporting. The system leverages ContikiNG and Cooja for simulating IoT data generation, Node-RED for data processing, and Spark for advanced analytics. The project tracks groups of people carrying IoT devices, identifies when a group forms or changes, and reports these events to a backend for further analysis.

1.2 Project Overview

In this project, people equipped with IoT devices detect proximity to one another using radio signals. A "group" is formed when at least three people are in contact, meaning each person is within the radio range (1-hop distance) of the other two. Once a group is created or changes in size (due to a person joining or leaving), a notification containing the updated group cardinality is sent to the backend. The backend periodically computes statistics on groups, including their lifetime, average, maximum, and minimum cardinality. Additionally, advanced analytics focus on participants' nationality and age, providing insights like a 7-day moving average of participants' ages per nationality, percentage increases in these averages, and the top nationalities with the highest percentage increases over time.

1.3 Assumptions and Guidelines

- The IoT devices are continuously reachable, possibly via multiple hops, through a static IoT device that serves as an IPv6 border router. Network partition cases are not considered.
- The IoT part of the system can be developed and tested using the COOJA simulator, where mobility is simulated by manually moving nodes.
- Group monitoring, including membership changes, is implemented using periodic functionality, with people leaving a group managed via a timeout mechanism.
- The backend handles statistical computations and advanced analytics, such as 7-day moving averages for participants' ages by nationality and related percentage changes.
- The backend identifies the top nationalities with the highest percentage increase, with the top N capped at 3 by default, though this parameter can be adjusted in the code.
- No advanced networking failures, such as partitioning, are accounted for in this implementation.

2 System Architecture

2.1 High-Level Architecture Diagram

The architecture for the Group Monitoring in Mobile IoT project is designed to track and manage the formation, evolution, and dissolution of groups of individuals carrying IoT devices. This system employs a multi-layered framework consisting of IoT devices, a Node-RED server, and an Apache Spark back-end, all of which work together to deliver real-time analytics. The high-level architecture is illustrated in Figure 1.

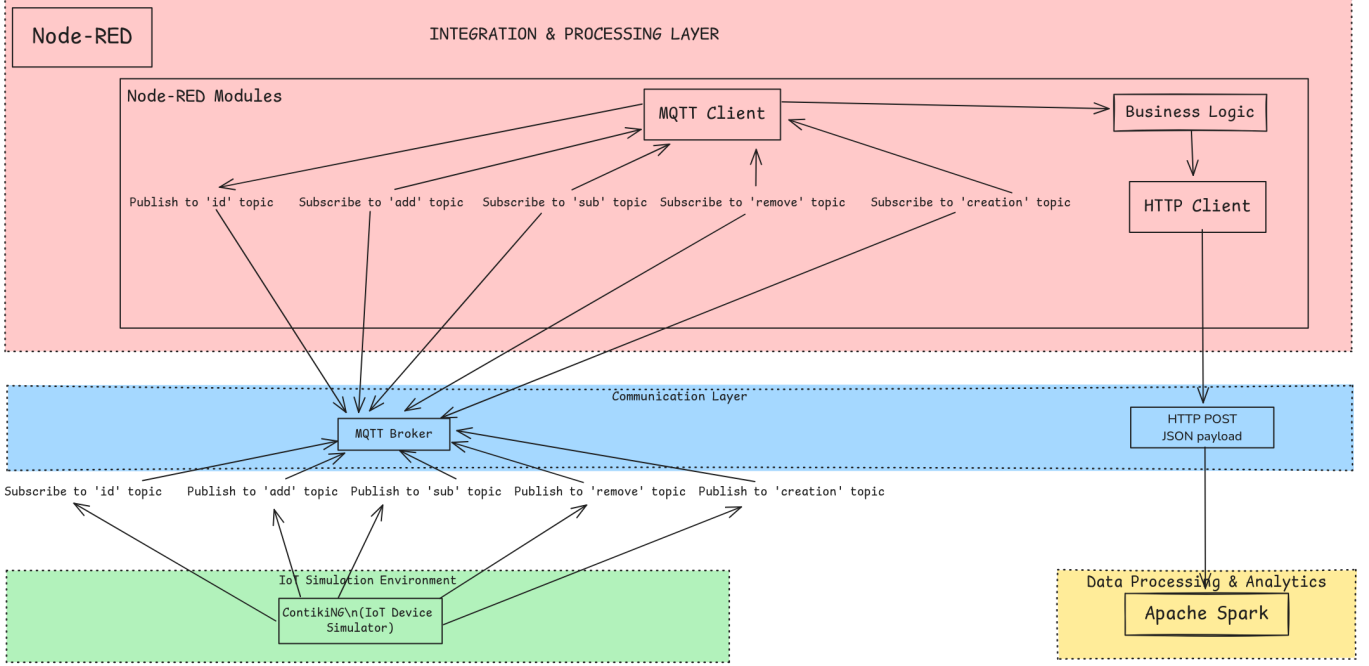


Figure 1: High-level architecture

2.2 Communication Protocols

The system uses both MQTT and REST protocols to enable communication across its three services. IoT devices communicate with the Node-RED server via MQTT, which follows a publish/subscribe model. This model decouples the sender (publisher) from the receiver (subscriber) using a message broker, which filters incoming messages from publishers and distributes them appropriately to subscribers. The Node-RED server communicates with the Apache Spark server through REST APIs, sending JSON-formatted participant data for further processing. Analysts can then access the processed data by performing HTTP requests to the Spark server.

This architecture ensures efficient real-time monitoring of group dynamics in mobile IoT environments, with robust data processing and analytics capabilities provided by Apache Spark.

3 IoT Simulation Environment (Contiki, Cooja)

3.1 System Constraints

Due to the resource constraints of the devices in the network, we simulate this limitation by imposing fixed memory capacities. Each device is only responsible for storing data pertinent to its own functionality. Any irrelevant messages are discarded.

3.2 System Implementation

The system is implemented in two main phases: initialization and operation.

1. **Initialization Phase:** During this phase, each device:

- (a) Configures the necessary data structures.

- (b) Establishes a connection with an MQTT broker through the MQTT protocol.
 - (c) Prepares to publish data once the connection is established.
2. **Operation Phase:** After initialization, devices begin interacting with their environment. Each device periodically initiates a discovery process to identify reachable neighbors. The steps in this process are as follows:
- (a) The device broadcasts a discovery message to detect nearby devices.
 - (b) Neighboring devices respond by sending their unique IDs, which are logged in the device’s neighbor list.
 - (c) The device requests the neighbor lists from its detected neighbors.
 - (d) Every time a list is received, the common neighbors between the local proximity list and the received list are computed. If the number of common neighbors is greater than 2, the receiver node computes all possible combinations of groups that can be created from these nodes, including itself. It then stores all possible groups, ordered by cardinality.

Once the grouping phase ends, the node evaluates potential groups and promotes them into actual groups. If a group is formed, or if existing groups change, the node with the lowest ID in the group acts as the leader and sends notifications to the backend. If the leader is a newly joined node, it will send a group creation message to the backend, which will then detect and respond to the situation appropriately.

3.3 Limitations

This approach has limitations in detecting multiple group changes between computations. For instance, if a group changes from (3, 5, 6, 7) to (3, 5, 7, 8), the system treats this as a new group and deletes the old one. This behavior arises because we rely on periodic updates from neighbor lists and cannot track every change in real-time. Consequently, our strategy focuses on detecting only monotonic changes, such as additions or deletions. If a group gains new members, the previous group is entirely included in the new one, and we trigger an addition event. Similarly, if nodes are removed from a group, we recognize it as a deletion event.

3.4 System Scenario Overview

The system manages four primary scenarios related to group management:

- Formation of a new group.
- Expansion of an existing group.
- Reduction in the size of an existing group.
- Removal of a group when it ceases to exist.

Scenario	Topic	Leader’s Role
New Group Formation	creation	The leader initiates a notification to announce the new group.
Group Expansion	add	The leader sends an update before the new member is added.
Group Reduction	sub	The leader sends an update when a member is removed.
Group Removal	remove	The leader sends a notification.

Table 1: Group Management Scenarios

4 Integration & Processing Backend (Node-RED)

4.1 High Level Architecture

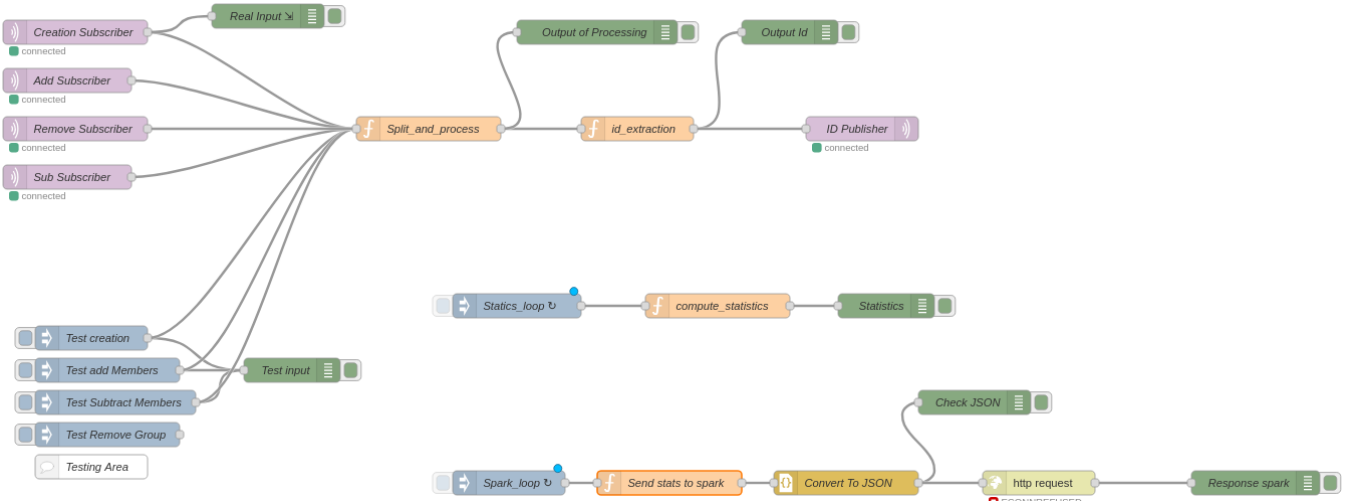


Figure 2: High Level Architecture of Nodes

4.2 Design Choices

For the communication between IoT devices and the Node-RED server, MQTT has been chosen over the standard HTTPS protocol due to its publish/subscribe architecture, which offers several advantages:

- **Topic-based filtering:** MQTT allows devices to subscribe only to specific topics, ensuring they receive only relevant messages. This minimizes unnecessary data overhead.
- **Decoupled communication:** MQTT enables the separation of publishers (devices or systems sending data) and subscribers (devices or systems receiving data). These components can operate independently without requiring direct knowledge of each other, making it ideal for dynamic IoT environments with frequently changing devices.
- **Asynchronous communication:** The publish/subscribe model supports asynchronous data exchange.
- **Simplified architecture via message broker:** The use of an MQTT broker centralizes the management of communication, offloading the complexity from individual devices. The broker handles message forwarding to subscribers, reducing the communication management burden on devices and simplifying system architecture overall.

4.3 Implementation of Group Monitoring

In this project, Node-RED is used as the central hub for managing MQTT communication between IoT devices and processing the data to determine group dynamics. The Node-RED flow consists of several key nodes configured to handle different MQTT topics and process messages accordingly. The architecture is designed to manage the creation, addition, and removal of group members and compute basic statistics on group memberships.

4.3.1 MQTT Communication Nodes

Table 2 lists all MQTT nodes along with their respective roles and topics. The format for both sent and received messages follows the structure shown below:

`<id>;<node_1>, ... ,<node_n>`

Here, `id` represents the group identifier, while `node_1` through `node_n` represent the device IDs that make up the group.

Node Name	Role	Topic	Description
Creation Subscriber	Subscriber	creation	Receives a message formatted as in 4.3.1, with the <code>id</code> field always set to <code>-1</code> . It instructs the "Split_and_process" node to create a new group based on the devices listed in the message.
Add Subscriber	Subscriber	add	Receives a message formatted as in 4.3.1, where <code>id</code> specifies the group to modify, and the nodes represent its current members. It notifies the "Split_and_process" node to update the group with the new members provided.
Remove Subscriber	Subscriber	remove	Receives a message formatted as in 4.3.1, where <code>id</code> specifies the group to delete, and the nodes represent its current members. It notifies the "Split_and_process" node to remove the group provided.
Sub Subscriber	Subscriber	sub	Receives a message formatted as in 4.3.1, where <code>id</code> specifies the group to modify, and the nodes represent its current members. It instructs the "Split_and_process" node to subtract (i.e. remove) the members listed in the message from the group.
Id Publisher	Publisher	id	Sends a message formatted as in 4.3.1 when a new group is created, where <code>id</code> represents the new group ID, and the nodes represent its current members. This information is sent back to the Contiki environment to ensure unique IDs for each group.

Table 2: MQTT Input and Output Nodes

4.3.2 Function Nodes for Data Processing

This section outlines the key functions responsible for data processing within the Node-RED environment. These functions are implemented within Node-RED function nodes, which are used to execute custom JavaScript code in response to incoming MQTT messages. These functions save the state, which needs to be maintained across multiple nodes within a flow, into the "flow context".

We focus on two primary functions: one for creating a new group and another for adding a member to an existing group. Additional functions, such as those for group removal and member subtraction, follow a similar pattern and are not included here for brevity. You can refer to the project repository for further details.

- **handleNewGroup:** This function manages the creation of new groups by parsing the input data, generating a unique group ID, initializing the group data structure, saving it in the flow context, and emitting an event to report the formation of the group.

```
function handleNewGroup(inputData) {
  // Parse inputData to extract group members
  members = parseMembers(inputData);

  // Generate unique group ID and current time
  groupId = generateGroupId();
  currentTime = getCurrentTime();

  // Initialize group data structure
  groupData = {
    id: groupId,
```

```

        time: currentTime,
        cardinality: members.length,
        members: members,
        statistics: {
            lifetimeStart: currentTime,
            cardinalityHistory: [members.length]
        }
    };

    saveGroupData(groupId, groupData); // Store group data in the flow context

    emitEvent('group_formed', groupData);
}

```

- **handleJoin:** This function handles adding a new member to an existing group. It updates the group's cardinality and statistics, and emits an event to signal that a new member has joined.

```

function handleJoin(inputData) {
    groupId = inputData.group_id;
    personData = inputData.person;

    group = getGroupData(groupId); // Retrieve group information

    if (group) {
        // Update group members and cardinality
        group.members.push(personData);
        group.cardinality += 1;
        group.statistics.cardinalityHistory.push(group.cardinality);

        saveGroupData(groupId, group); // Update group data in flow context

        emitEvent('member_joined', group);
    }
}

```

4.3.3 Periodic Computation

Certain nodes perform periodic computations and send statistics or group-related information, as summarized in Table 3.

Node Name	Timer	Destination Node	Description
Statics_loop	5 minutes	Compute_statistics	Triggers the destination node to compute updated statistics, including group lifetime and minimum/maximum cardinality.
Spark_loop	Every 24 hours (temporarily set to 2 minutes for testing)	Send_stats_to_spark	Triggers the destination node to generate and send a JSON object containing user nationality and age data to Spark.

Table 3: Periodic nodes

4.4 Integration with Apache Spark Streaming

Once data from IoT devices has been processed and aggregated by Node-RED, it is forwarded to an Apache Spark Streaming back-end for advanced analytics (e.g., a 7-day moving average of participants' age per nationality). The data is sent in JSON format to a specified endpoint provided by the Spark server. For more information on how the data is received by the Spark server and details about the endpoint configuration, please refer to the following section.

5 Analytics Backend (Spark)

5.1 High-Level Architecture Diagram

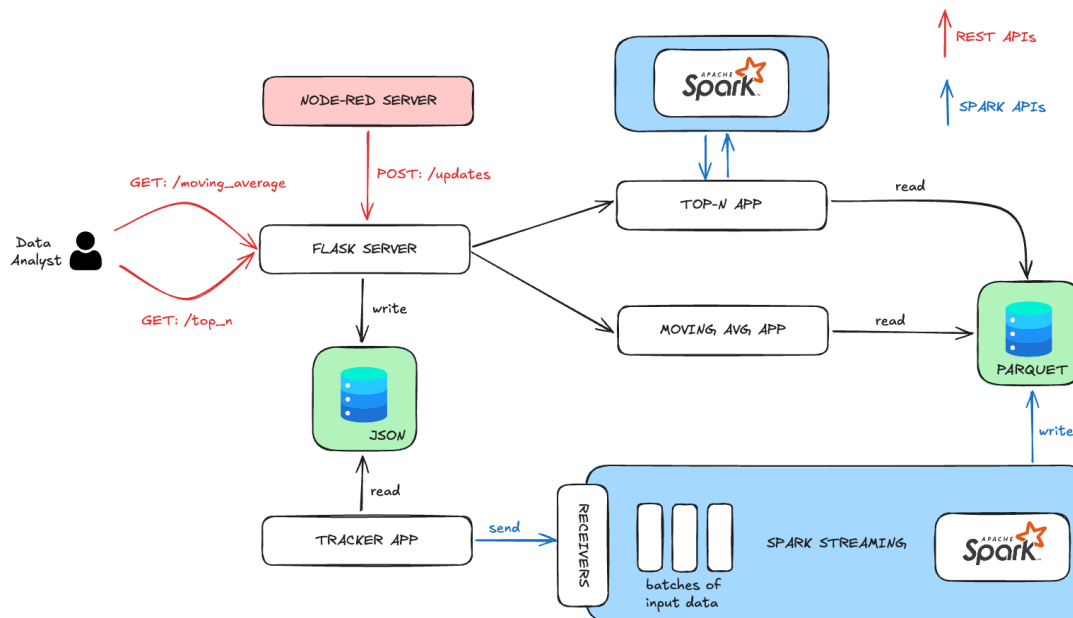


Figure 3: High-level Architecture of the Analytics System

5.2 Design Choices

Two key design choices were made: one for the Tracker App and another for the TopN app. The Tracker App utilizes Spark Structured Streaming due to its support for continuous processing, allowing data to be processed incrementally as it arrives. This enables the calculation of a moving average in real-time as new data is received, which is particularly useful when outputting results to the console.

However, for the TopN App, the output from the Tracker App could not be directly printed to the screen. Instead, it needed to be stored in Parquet files. The reason for this is that TopN requires ranking the processed data, but as of September 2024, the Python Structured Streaming API does not offer built-in ranking functionality. As noted in the documentation under the "Unsupported Operations" section: "*Sorting operations are supported on streaming Datasets only after an aggregation and in Complete Output Mode*" [1]. This means that the only way to rank the data is by using standard SQL operations in Spark. Therefore, the output of the processed streaming data had to be stored as Parquet files in local storage. To observe the percentage increase, the data must be fetched through a separate terminal. To compute the TopN, the stored data needs to be retrieved, processed by Spark again to sort it, and then printed in a different terminal.

5.3 Component Description

5.3.1 Entry Point

The system's entry point is the `main.py` file. This script initializes both the Flask server and the tracker application based on Structured Streaming, running them on separate threads to ensure they operate concurrently.

5.4 Flask Server

The server is implemented using Python's Flask framework, with the core logic residing in the `app.py` file. This file defines three REST endpoints, which are detailed in Table 4.

Endpoint	Method	Description	End-User
/updates	POST	Receives updates from the Node-RED server, which must follow the format specified in Table 5. The server stores each update in real time in a uniquely identified file, allowing Spark Streaming to process and analyze the data as it arrives.	Node-RED Server
/moving_average	GET	Executes the <code>moving_average.py</code> script, which retrieves Parquet files containing processed data (moving averages and percentage increases) and displays the results in the console.	Data Analyst
/top_n	GET	Executes the <code>top_n.py</code> script, which retrieves Parquet files containing processed data (moving averages and percentage increases), ranks nationalities (top 3) based on the percentage increase for each day, and displays the results in the console.	Data Analyst

Table 4: REST endpoints exposed by the Spark server

Field Name	Type	Level	Description
timestamp	String	Root Level	The time at which the data was recorded, formatted as "YYYY-MM-DD HH:MM:SS"
participants	Array of Objects	Root Level	A list of participants, each represented as an object with the fields described in Table 6

Table 5: Root-Level Data Format Stored by `app.py`

Field Name	Type	Level	Description
device_id	Integer	Participants Object	Unique identifier for the participant's device
nationality	String	Participants Object	Nationality of the participant
age	Integer	Participants Object	Age of the participant

Table 6: Structure of Participant Objects

5.5 Tracker App

The Tracker App utilizes Apache Spark's Structured Streaming API to process data in real-time. It ingests JSON files as a stream, then explodes the participants array to generate one row per participant. This data is subsequently grouped by nationality and windowed over a 7-day period, with a sliding interval of 1 day. The window partitioning configuration includes a watermark duration, which defines the maximum delay the system will tolerate for late data. Specifically, any data arriving later than the specified watermark duration (5 minutes in this case) after the event time will be considered late and discarded. This helps manage the trade-off between processing late data and the performance of the streaming application. Refer to the Table 7 for a summary of the configuration.

Configuration Parameter	Value	Description
Watermark Duration	5 minutes	Allows Spark to handle late-arriving data up to a certain time.
Window Duration	7 days	Time window over which the moving average is calculated.
Slide Duration	1 day	Frequency at which the window slides, generating a new result.

Table 7: Spark Streaming Configuration Parameters

Within each group, the average age is calculated using the following code:

```
averaged_data = processed_stream.groupBy(  
    window("timestamp", config["windowDuration"], config["slideDuration"]),  
    "nationality"  
) .avg("age")
```

To compute the daily percentage increase of the 7-day moving average of age, we need to align the data by nationality, as we are interested in calculating the percentage increase for each nationality separately. Additionally, we must align the time windows by offsetting the start time by the slide duration, which in this case is 1 day. This allows us to compare the 7-day average of one window to that of the previous window, which is shifted by 1 day. To simplify the process, we create two alias tables: "yesterday" for the previous window and "today" for the current window. The join condition, excluding Spark-specific syntax, can be expressed as:

```
today.window.start = yesterday.window.start + slideDuration  
  
AND  
  
today.nationality = yesterday.nationality
```

For easier implementation, a column is added that represents the window start time plus the slide duration. This makes the join operation straightforward. The resulting code looks like this:

```
averaged_data = averaged_data.withColumn(  
    "window_start_plus_slide",  
    expr(f"window.start + INTERVAL {config['slideDuration']}")  
)  
percentage_increase = averaged_data.alias("today").join(  
    averaged_data.alias("yesterday"),  
    (col("today.window.start") == col("yesterday.window_start_plus_slide")) &  
    (col("today.nationality") == col("yesterday.nationality"))  
)
```

This join operation aligns each day's data with that of the previous day, enabling the calculation of the percentage increase using the formula:

$$\text{Percentage Increase} = \left(\frac{\text{today.avg(age)} - \text{yesterday.avg(age)}}{\text{yesterday.avg(age)}} \right) \times 100$$

In the code, the final calculation is performed as follows:

```
percentage_increase = percentage_increase.select(  
    col("today.nationality"),  
    col("yesterday.window.start").alias("prev_start_time"),
```

```
col("yesterday.window.end").alias("prev_end_time"),
col("yesterday.avg(age)").alias("prev_avg_age"),
col("today.window.start").alias("start_time"),
col("today.window.end").alias("end_time"),
col("today.avg(age)").alias("avg_age"),
((col("today.avg(age)") - col("yesterday.avg(age)")) /
 col("yesterday.avg(age)") * 100).alias("percentage_increase")
)
```

The final output is stored locally in Parquet files, which can be retrieved and written to a file in a human-readable table format by performing a GET request to the `/moving_average` endpoint.

5.6 Top-N App

The Top-N App is responsible for ranking the top three nationalities based on the daily percentage increase in the 7-day moving average of participant ages. This app operates by reading the processed Parquet output generated by the Tracker App, further processing it in real-time to produce a ranked list of nationalities for each each day.

5.6.1 Functionality Overview

When a GET request is made to the `/top_n` endpoint, the Top-N App performs the following steps:

1. **Reading Data:** Reads the percentage increase data from the Parquet files generated by the Tracker App.
2. **Partitioning:** Partitions the data into 1-day windows based on the `start_time` column to allow the application to rank nationalities within each time frame.

```
slided_window = Window.partitionBy(window(col("start_time"), CONFIG["
    slideDuration"])).orderBy(desc("percentage_increase"))
```

3. **Ranking:** Within each 1-day window, the nationalities are ordered by their percentage increase, and a rank is assigned using the `row_number()` function. This rank determines the position of each nationality relative to others in terms of percentage increase.

```
ranked_data = percentage_increase_df.withColumn("rank", row_number().over(
    slided_window))
```

4. **Filtering:** Filters the ranked data to retain only the top three nationalities for each window.

```
top_nationalities = ranked_data.filter(col("rank") <= 3)
```

5. **Output:** Extracts and writes fields such as time window, nationality, and rank to a file in a human-readable table format.

6 Conclusion

6.1 Summary

The Group Monitoring System for Mobile IoT devices was developed utilizing a technology stack that includes Contiki, Cooja, Node-Red (JavaScript), and Apache Spark (Python). The system simulates the exchange of IoT data, which is then transmitted to a server where it undergoes processing using distributed computing technologies. Each layer of the system is deployed on a separate Virtual Machine, ensuring modularity.

6.2 Future Work

For future development, Apache Spark could be deployed across multiple machines to accelerate computations. A single machine deployment may be insufficient for processing large amounts of data. By utilizing a cluster of computers, such as through services like Amazon EMR, the system can scale to meet increasing data demands.

References

- [1] PySpark Structured Streaming Documentation. "Unsupported Operations". In: (2024). URL: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#unsupported-operations>.